Lezione 4

Architettura MVP e Containerizzazione con Docker

Simone Colli

31 ottobre 2025

Indice

1. Contenuto della lezione
2. Pattern architetturali
3. Architettura MVP nel progetto
4. Introduzione a Docker
5. Struttura del progetto con Docker
6. Best Practices

Contenuto della lezione

- Pattern architetturali: MVC vs MVP
- Riorganizzazione del progetto in architettura MVP
- Separazione delle responsabilità: Model, View, Presenter
- Introduzione a Docker: containerizzazione
- Dockerfile: creazione immagini
- Setup ambiente di sviluppo con Docker
- Struttura repository multi-servizio

Perché serve un'architettura?

Fino alla lezione precedente abbiamo scritto codice funzionante, ma senza una **struttura chiara**.

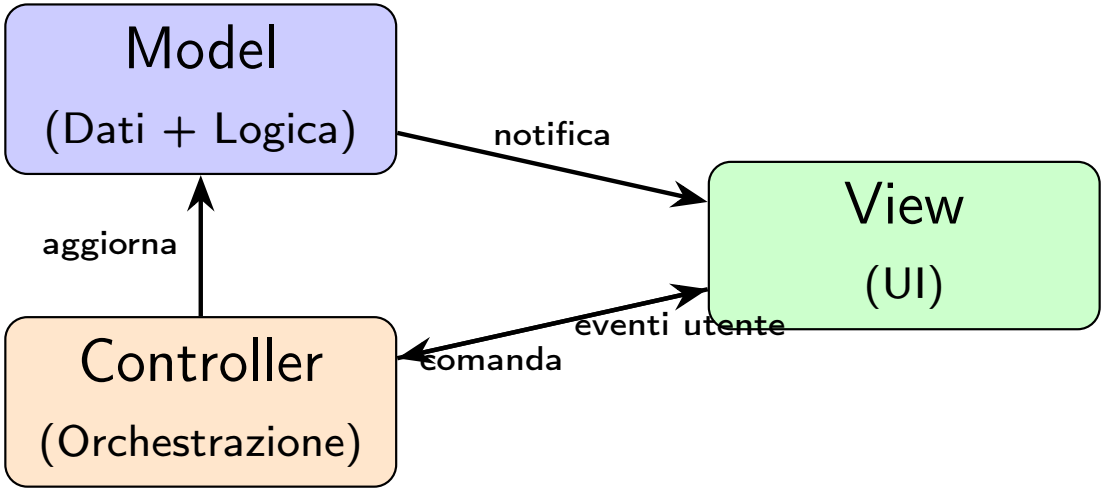
Problemi del codice "spaghetti"

- **Difficile manutenzione**: logica sparsa ovunque
- **Testing complesso**: dipendenze strette tra componenti
- **Riusabilità bassa**: componenti troppo accoppiati
- **Scalabilità limitata**: aggiungere feature diventa complicato

Soluzione: applicare un **pattern architetturale** che separi le responsabilità.

MVC - Model View Controller

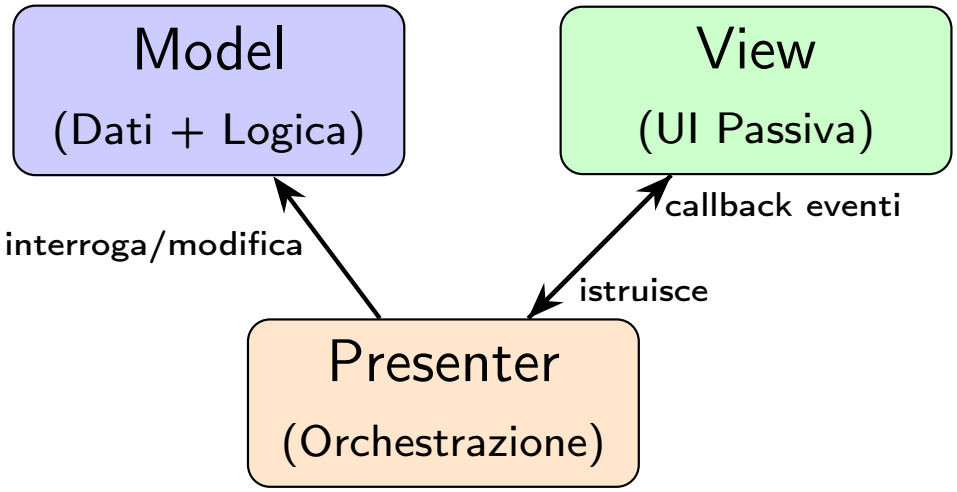
MVC è uno dei pattern più diffusi per organizzare applicazioni interattive.



- **Model**: gestisce dati e business logic
- **View**: mostra l'interfaccia e cattura input
- **Controller**: riceve input, aggiorna Model, comanda View

MVP - Model View Presenter

MVP è una variante di MVC dove la View è completamente **passiva**.



Differenza chiave

Nel MVP, la View **non conosce** il Model. Ogni aggiornamento passa attraverso il Presenter.

Vantaggio: View completamente testabile in isolamento.

MVC vs MVP - Confronto

Aspetto	MVC	MVP
View	Osserva il Model (binding bidirezionale)	Completamente passiva, non conosce Model
Controller/ Presenter	Gestisce input utente	Gestisce input E istruisce la View
Testabilità	Media (View dipende da Model)	Alta (View isolata)
Complessità	Più semplice per app piccole	Più verboso ma più scalabile
Uso tipico	Web framework (Rails, Django)	App enterprise, mobile

Scelta per il nostro progetto: MVP, per massimizzare separazione e testabilità.

Struttura del progetto MVP

Riorganizziamo il codice in **tre strati** ben separati:

Model Logica di dominio e accesso ai dati

- `Task.js`: entità di dominio
- `TaskList.js`: collezione di task (business logic)
- `Task2localStorage.js`: persistenza dati

View Interfaccia utente passiva

- `AppView.js`: controlli principali (bottoni)
- `TaskListView.js`: rendering lista task
- `TaskView.js`: rendering singolo task
- `ModalView.js`: form di inserimento/modifica

Presenter Orchestrazione e logica di presentazione

- `AppPresenter.js`: coordina tutto, gestisce eventi

Model - Responsabilità

Il **Model** incapsula la **business logic** e l'accesso ai dati.

Responsabilità del Model

- **Validazione**: regole di dominio (es. data valida)
- **Stato**: gestione dello stato dell'applicazione
- **Persistenza**: caricamento/salvataggio dati
- **Logica di business**: calcoli, ordinamenti, filtri

Il Model NON deve:

- Conoscere la View o il DOM
- Gestire eventi dell'utente
- Decidere cosa mostrare a schermo

View - Responsabilità

La **View** è **completamente passiva**: riceve istruzioni e notifica eventi.

Responsabilità della View

- **Rendering**: mostrare dati ricevuti dal Presenter
- **Cattura eventi**: delegare eventi al Presenter (callback)
- **Manipolazione DOM**: creazione/aggiornamento elementi

La View NON deve:

- Contenere logica di business
- Accedere direttamente al Model
- Prendere decisioni (es. validazione)
- Fare richieste HTTP

Pattern: la View espone `bind*(handler)` per collegare callback del Presenter.

Presenter - Responsabilità

Il **Presenter** è il **cervello** dell'applicazione: orchestrazione pura.

Responsabilità del Presenter

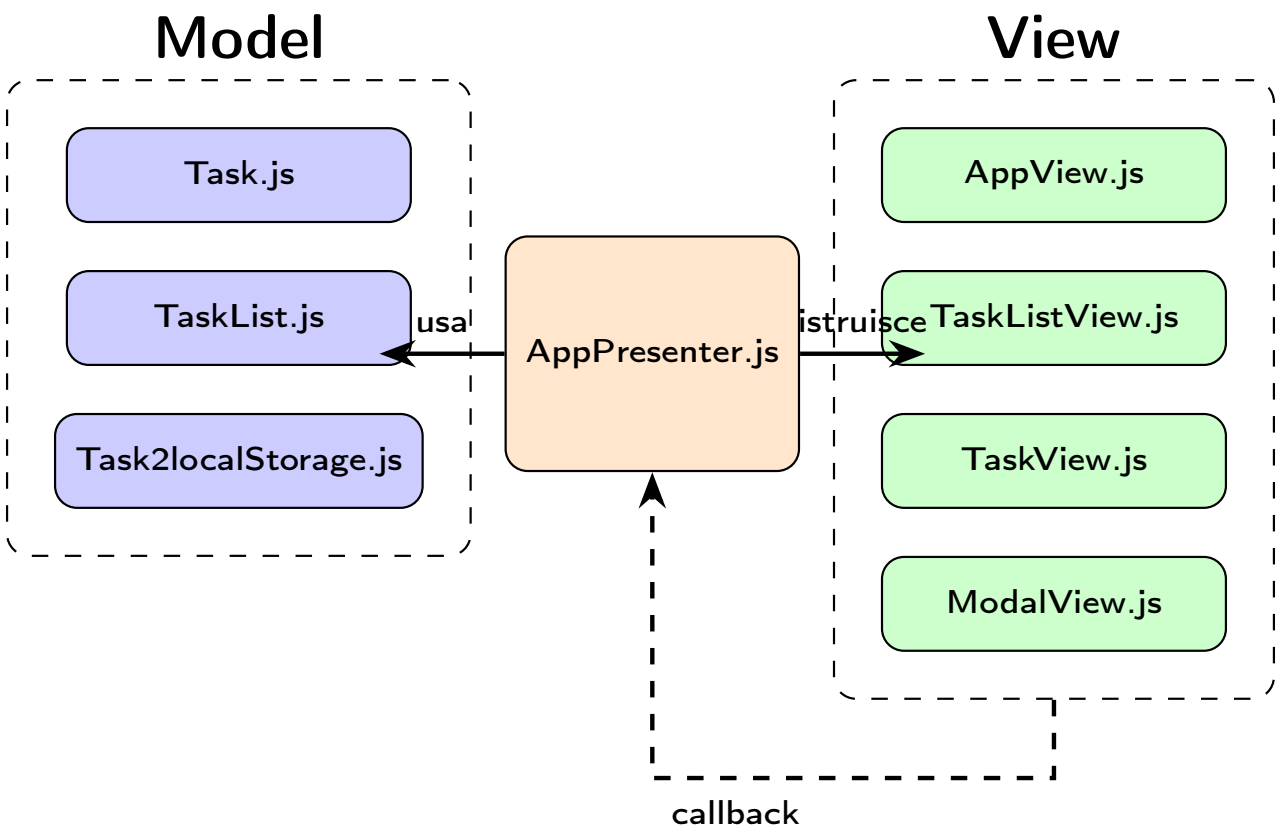
- **Gestione eventi**: riceve callback dalla View
- **Orchestrazione**: coordina Model e View
- **Logica di presentazione**: formattazione, conversioni
- **Flusso applicazione**: ordine operazioni, validazioni UI

Pattern: il Presenter **non** fa richieste HTTP direttamente, le delega al Model.

Esempio di flusso:

1. View notifica: "utente ha cliccato Aggiungi"
2. Presenter: valida input, crea Task, chiama Model
3. Model: salva dati
4. Presenter: istruisce View a mostrare successo

Diagramma dell'architettura MVP completa



Flusso:

- View cattura evento → callback Presenter
- Presenter interroga/modifica Model
- Presenter istruisce View su cosa mostrare

Vantaggi dell'architettura MVP

Testabilità

- **Model**: testabile in isolamento (unit test)
- **View**: mockabile facilmente
- **Presenter**: testabile con mock di Model e View

Manutenibilità

- Responsabilità chiare: facile individuare dove modificare
- Cambio UI: modifica solo View, Presenter e Model invariati
- Cambio logica: modifica solo Model

Riusabilità

- Model riutilizzabile in altri contesti (es. mobile app)
- View sostituibile (es. passaggio da jQuery a React)

Cos'è Docker?

Docker è una piattaforma per sviluppare, distribuire ed eseguire applicazioni in **container**.

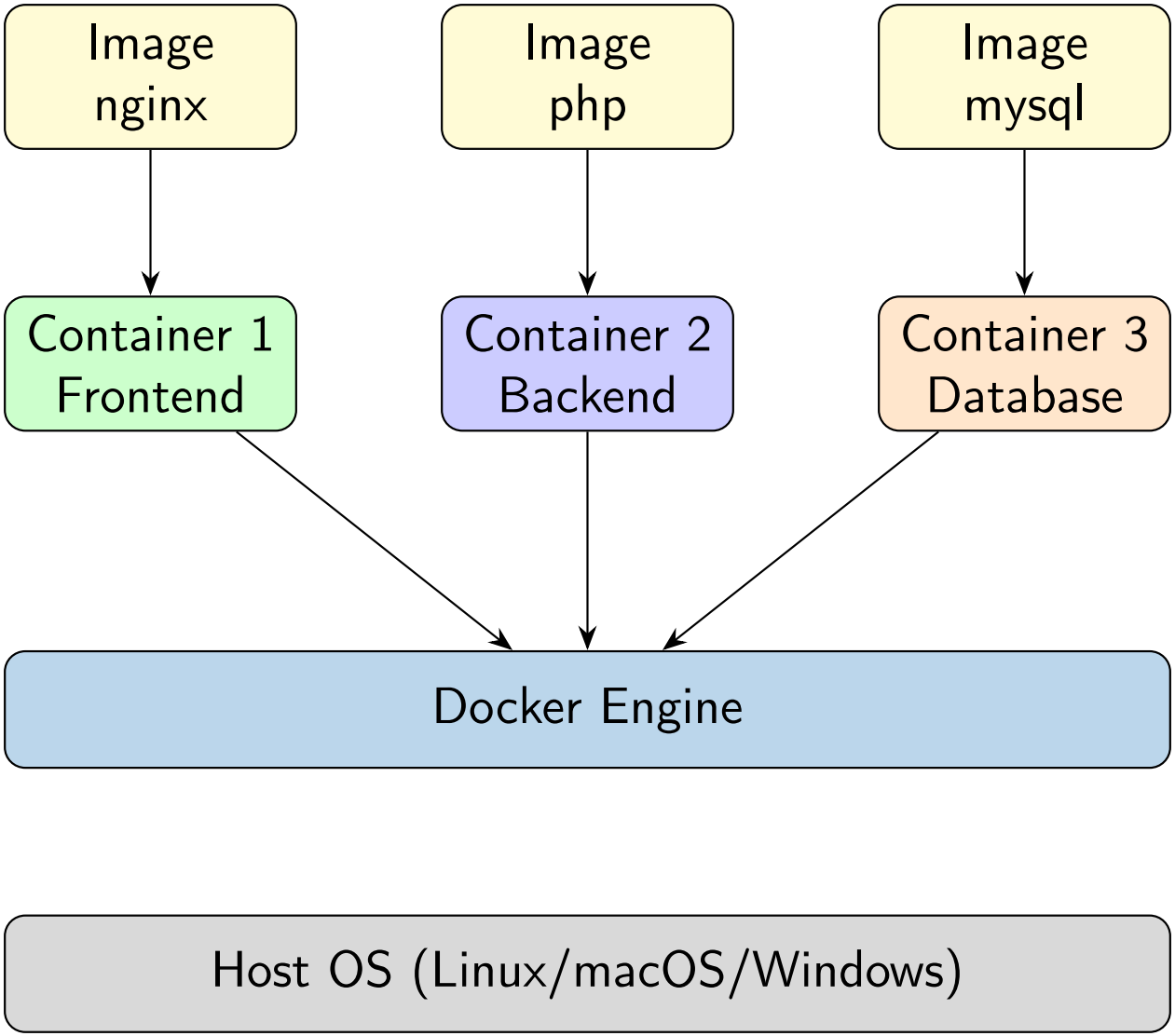
Container vs Virtual Machine

- **VM**: virtualizza l'intero sistema operativo (pesante)
- **Container**: isola l'applicazione condividendo il kernel dell'host (leggero)

Vantaggi:

- **Portabilità**: "funziona sul mio PC" $\rightarrow$ funziona ovunque
- **Isolamento**: ogni servizio in un container separato
- **Riproducibilità**: ambiente identico per tutti i developer
- **Scalabilità**: facile replicare container

Architettura Docker



Concetti chiave Docker

Dockerfile File di testo con istruzioni per costruire un'immagine

Image Template read-only con l'applicazione e dipendenze

Container Istanza runtime di un'immagine (processo isolato)

Volume Storage persistente esterno al container

Network Rete virtuale per comunicazione tra container

Docker Hub Registry pubblico di immagini (come GitHub per codice)

Docker Compose Tool per orchestrare multi-container

Struttura repository multi-servizio

```
Lezione_04/
|-- services/
|   |-- frontend/
|   |   |-- Dockerfile.development
|   |   |-- Dockerfile.production
|   |   |-- src/              # codice HTML/CSS/JS
|   |-- backend/
|   |   |-- Dockerfile.development
|   |   |-- Dockerfile.production
|   |   |-- src/              # codice PHP/API
|   |-- database/
|       |-- Dockerfile.development
|       |-- Dockerfile.production
|       |-- configs/          # script SQL iniziali
|-- config/
|   |-- apache/               # configurazioni Apache
|   |-- start.sh              # script avvio servizi
|-- storage/                  # dati persistenti (gitignore)
|-- Dockerfile.simple         # versione monolitica (didattica)
```

Best Practices Docker

Dockerfile

- **Layer caching**: ordina istruzioni per ottimizzare cache
- **Multi-stage builds**: separa build e runtime
- **.dockerignore**: escludi file non necessari
- **Minimizza layer**: raggruppa `RUN` quando possibile
- **User non-root**: non eseguire app come root in produzione

Architettura

- **Un processo per container**: facilita debugging e scaling
- **Stateless containers**: stato esterno (volume/DB)
- **Variabili d'ambiente**: configurazione iniettata
- **Health checks**: verifica stato del servizio

Best Practices MVP

Separazione delle responsabilità

- **View**: zero logica, solo rendering e binding
- **Model**: business logic, non conosce View
- **Presenter**: orchestrazione, nessun accesso diretto al DOM

Testing

- **Model**: unit test puri (no dipendenze esterne)
- **Presenter**: test con mock di Model e View
- **View**: test di integrazione con DOM reale o virtuale

Manutenibilità

- **Dependency Injection**: passare dipendenze via costruttore
- **Naming conventions**: `handle*` per Presenter, `bind*` per View
- **Single Responsibility**: ogni classe ha un unico scopo